AGENTS.md

Coworking Space Platform — AI Development Rules

1. Project Overview

Build and maintain an enterprise-grade coworking-space management platform using:

- **Frontend:** Next.js App Router
- **Backend:** NestJS
- **Database:** MySQL
- **ORM:** Prisma
- **Runtime:** Node.js 20 LTS
- **Language:** TypeScript
- **Styling:** Tailwind CSS
- **UI:** shadcn/ui + Magic UI + HeroUI + Tremor + Framer Motion
- **Validation:** Zod and/or class-validator
- **Real-time:** NestJS WebSockets + Socket.io
- **Payments:** Razorpay
- **Email:** NestJS Mailer + Nodemailer SMTP

The application must be designed as a modular, scalable, maintainable production-grade system.

Do not build the application as a collection of tightly coupled pages, controllers, services, or utility files.

Every feature must have clear ownership and boundaries.

---

2. Core Development Principles

Every implementation must follow these principles:

1. **Modularity**
2. **Single Responsibility**
3. **Feature Isolation**
4. **Strong Type Safety**
5. **Reusable Components**
6. **Secure-by-default implementation**
7. **Database integrity**
8. **Consistent API contracts**
9. **Accessible UI**
10. **Responsive design**
11. **Testability**
12. **Maintainability**
13. **Performance**

14. **Clear error handling**
15. **No unnecessary duplication**

Never sacrifice architecture merely to make a feature work quickly.

---

3. Technology Version Locks

Use the following technology baseline:

Technology	Required
Next.js	14.2+ or 15.x
Next.js Router	App Router
NestJS	10.x
Node.js	20 LTS
TypeScript	5.4+
MySQL	8.0+
Prisma	5.14+
Tailwind CSS	4.x
Socket.io	4.7+
Razorpay SDK	2.9+
Nodemailer	6.9+
Zod	3.23+
class-validator	Compatible latest version

The original workspace specification explicitly locks these technologies and versions.

Do not introduce an alternative framework or ORM without explicit approval.

---

4. Repository Structure

Use a monorepo-style structure:

```
/
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   ├── features/
```

```
│   │   ├── components/
│   │   │   └── ui/
│   │   ├── lib/
│   │   ├── hooks/
│   │   ├── providers/
│   │   ├── types/
│   │   └── styles/
│   ├── public/
│   ├── package.json
│   ├── tsconfig.json
│   └── next.config.*
│
├── backend/
│   ├── src/
│   │   ├── modules/
│   │   ├── shared/
│   │   ├── config/
│   │   ├── app.module.ts
│   │   └── main.ts
│   ├── prisma/
│   │   ├── schema.prisma
│   │   ├── migrations/
│   │   └── seed.ts
│   ├── test/
│   ├── package.json
│   └── tsconfig.json
│
├── docs/
├── .env.example
├── AGENTS.md
└── README.md
```

If the existing repository already uses a different top-level structure, do not blindly restructure it.

First inspect the repository and preserve working infrastructure unless there is a strong architectural reason to change it.

---

5. Frontend Architecture

The frontend must use the Next.js App Router.

```
frontend/src/

├── app/
│   ├── (public)/
│   ├── (auth)/
│   ├── dashboard/
│   ├── admin/
```

```
│   ├── bookings/
│   ├── meeting-rooms/
│   ├── community/
│   ├── wallet/
│   ├── billing/
│   ├── cms/
│   ├── layout.tsx
│   └── page.tsx
│
├── features/
│   ├── auth/
│   ├── bookings/
│   ├── meeting-rooms/
│   ├── floor-map/
│   ├── wallet-billing/
│   ├── community/
│   └── cms/
│
├── components/
│   └── ui/
│
├── lib/
├── hooks/
├── providers/
├── types/
└── styles/
```

The source specification requires feature-oriented frontend organization with thin App Router routes and self-contained domain features.

---

6. Frontend Feature Structure

Each feature should be self-contained.

Example:

```
features/meeting-rooms/

├── components/
├── hooks/
├── services/
├── store/
├── types/
├── schemas/
├── utils/
└── index.ts
```

Do not create:

```
components/MeetingRoomEverything.tsx
utils/allBusinessLogic.ts
services/everything.ts
```

Business logic belongs to the appropriate feature.

Shared functionality belongs in shared libraries only when it is genuinely reusable.

---

7. Next.js App Router Rules

Routes must remain thin.

A route should primarily:

- authenticate/authorize where appropriate
- load data
- compose feature components
- define metadata
- handle route-level concerns

Do not place large business logic directly inside:

```
page.tsx
layout.tsx
loading.tsx
error.tsx
route.ts
```

Business logic belongs in feature services/hooks or the backend.

Prefer Server Components by default.

Use Client Components only when required for:

- state
- browser APIs
- event handlers
- WebSockets
- interactive UI
- animations
- client-side libraries

Avoid unnecessary `"use client"`.

8. Backend Architecture

The backend must use NestJS modular architecture.

```
backend/src/

├── modules/
│   ├── auth/
│   ├── branches/
│   ├── bookings/
│   ├── meeting-rooms/
│   ├── floor-map/
│   ├── wallet-billing/
│   ├── community/
│   └── cms/
│
├── shared/
└── config/
```

This directly follows the required Modular NestJS Feature Pattern.

9. NestJS Module Structure

Each feature should follow:

```
modules/bookings/

├── bookings.module.ts
├── bookings.controller.ts
├── bookings.service.ts
├── dto/
│   ├── create-booking.dto.ts
│   ├── update-booking.dto.ts
│   └── booking-query.dto.ts
├── entities/
├── guards/
├── policies/
├── interfaces/
├── constants/
└── tests/
```

A feature may add additional folders when required.

Do not create unnecessary layers.

10. Feature Isolation

Feature modules must not directly access another feature's private implementation.

For example:

```
MeetingRoomsModule
```

must not directly access:

```
WalletBillingRepository
WalletBillingPrivateService
WalletBillingInternalEntity
```

Instead, communication must occur through:

- exported NestJS services
- shared interfaces
- controlled application services
- API contracts where appropriate

The source specification explicitly requires no cross-feature direct imports and independent upgradability.

11. Backend Controller Rules

Controllers must remain thin.

Controllers should:

1. receive request
2. validate request
3. authenticate user
4. authorize user
5. call service
6. return response

Controllers must NOT contain:

- complex business rules
- database queries
- payment logic
- booking conflict algorithms

- invoice calculations
- large transformations

Example:

```
@Post()
async createBooking(
  @CurrentUser() user: AuthUser,
  @Body() dto: CreateBookingDto,
) {
  return this.bookingsService.create(user.id, dto);
}
```

---

12. Service Rules

Services own business logic.

Services may:

- validate business rules
- call repositories/data access
- execute transactions
- coordinate feature operations
- publish events
- call external providers

Services should not become giant god classes.

If a service becomes too large, split responsibilities into focused services.

---

13. TypeScript Rules

TypeScript strict mode is mandatory.

```
{
  "compilerOptions": {
    "strict": true
  }
}
```

The original specification explicitly requires strict mode, explicit DTOs, request payloads, response interfaces and component props. It also prohibits `any`.

Never use:

```
any
```

Avoid:

```
as any
```

Avoid unnecessary type assertions.

Prefer:

```
unknown
```

with proper narrowing when the type is genuinely unknown.

---

14. DTO Rules

Every incoming API payload must have a DTO.

Example:

```
export class CreateBookingDto {
  @IsUUID()
  roomId!: string;

  @IsDateString()
  startTime!: string;

  @IsDateString()
  endTime!: string;
}
```

Validate all external input.

Use:

- class-validator + ValidationPipe
- or Zod

Do not trust frontend validation.

Backend validation is mandatory.

15. Global Validation

Configure NestJS validation globally.

Use:

```
new ValidationPipe({
  whitelist: true,
  forbidNonWhitelisted: true,
  transform: true,
});
```

Never allow arbitrary properties to silently enter application services.

16. API Versioning

Use versioned APIs.

Preferred structure:

```
/api/v1/auth
/api/v1/branches
/api/v1/bookings
/api/v1/meeting-rooms
/api/v1/floor-map
/api/v1/wallet
/api/v1/community
/api/v1/cms
```

Do not create inconsistent endpoint naming.

Use nouns rather than action-heavy URLs where practical.

Prefer:

```
POST /bookings
```

over:

```
POST /createNewBooking
```

17. API Response Standards

Successful responses should follow a predictable structure.

Example:

```
{
  "success": true,
  "data": {},
  "message": "Booking created successfully"
}
```

Errors must use the standardized structure:

```
{
  "success": false,
  "error": {
    "code": "SLOT_ALREADY_BOOKED",
    "message": "The selected time slot is no longer available.",
    "details": []
  }
}
```

This error structure is explicitly defined in the source specification.

18. Global Exception Handling

Use a centralized NestJS exception filter.

All expected application errors should expose:

```
success
error.code
error.message
error.details
```

Never expose:

- stack traces
- SQL queries
- database credentials
- internal filesystem paths
- secrets
- provider credentials

in production responses.

---

19. Swagger

Swagger must be enabled.

Controllers must use:

```
@ApiTags()
@ApiOperation()
```

Document:

- endpoints
- request DTOs
- responses
- authentication
- error responses

The source specification requires live Swagger documentation at:

```
/api/docs
```

---

20. Database

Use MySQL 8+ with Prisma.

Database access must go through Prisma.

Do not write raw SQL unless necessary.

When raw SQL is unavoidable:

- parameterize it
- document why it is required
- keep it inside the appropriate feature/data-access layer

21. Prisma Rules

Use Prisma migrations.

Never manually modify production database schema without a migration.

Every schema change must include:

```
prisma migrate
```

or an equivalent controlled migration process.

Keep seed data deterministic.

22. Database Integrity

Use database constraints wherever possible.

Examples:

- unique email
- unique booking reference
- unique room identifier
- foreign keys
- required fields
- indexes
- appropriate enum values

Do not rely solely on frontend checks for database integrity.

23. Transactions

Use Prisma transactions when multiple related database operations must succeed or fail together.

Examples:

- wallet deduction + transaction record
- booking + payment record
- refund + wallet credit
- invoice creation + payment state update

Never leave the database in a partially updated financial state.

24. Authentication

Authentication must be centralized in the Auth module.

The Auth feature owns:

- registration
- login
- logout
- password handling
- token handling
- session handling
- authentication guards
- user identity
- authorization primitives

Do not implement authentication independently inside other modules.

25. Authorization

Implement role-based access control.

At minimum support conceptual roles such as:

```
MEMBER
STAFF
MANAGER
ADMIN
SUPER_ADMIN
```

Do not hardcode role checks throughout the application.

Prefer reusable guards/policies.

Example:

```
@Roles(Role.ADMIN)
@UseGuards(JwtAuthGuard, RolesGuard)
```

Feature-specific permissions should be centralized.

26. Security

Security is mandatory.

Implement appropriate protections for:

- authentication
- authorization
- CORS
- CSRF where applicable
- rate limiting
- request validation
- secure cookies/tokens
- password hashing
- security headers
- file uploads
- webhook verification
- sensitive logging

Never commit:

```
.env
.env.local
.env.production
```

Never expose secrets in frontend code.

---

27. Environment Variables

Provide:

```
.env.example
```

Example:

```
DATABASE_URL=

JWT_SECRET=

RAZORPAY_KEY_ID=
RAZORPAY_KEY_SECRET=

SMTP_HOST=
SMTP_PORT=
SMTP_USER=
```

```
SMTP_PASSWORD=

NEXT_PUBLIC_API_URL=
```

Never put actual production secrets in the repository.

---

28. Meeting Room Module

The Meeting Rooms feature is responsible for:

- meeting room management
- room metadata
- capacity
- operating hours
- availability
- slot generation
- booking validation
- room status

The availability engine must be centralized inside this feature.

Do not duplicate availability calculations across frontend pages.

---

29. Booking Rules

Bookings must be validated on the server.

Before creating a booking:

1. authenticate user
2. validate room
3. validate requested time
4. validate operating hours
5. check availability
6. check conflicts
7. calculate price
8. process required payment/credits
9. create booking transactionally
10. emit relevant events
11. return booking confirmation

The frontend availability display is informational.

The backend is authoritative.

---

30. Double Booking Protection

The booking system must be resistant to concurrent requests.

Two users attempting to reserve the same room/time must not both successfully create conflicting bookings.

Use appropriate:

- database transactions
- locking
- conflict checks
- unique constraints where applicable

Never rely only on:

```
if (!existingBooking) {
  createBooking();
}
```

without concurrency protection.

---

31. Floor Map

The Floor Map feature owns:

- floor layouts
- desk/room positions
- visual availability
- interactive map
- live state
- WebSocket updates

Use Socket.io for real-time updates.

The backend gateway should publish meaningful domain events rather than arbitrary UI mutations.

---

32. WebSocket Rules

WebSocket events must be typed and documented.

Prefer:

```
room.status.updated
booking.created
booking.cancelled
desk.status.updated
```

over vague events such as:

```
update
change
refresh
```

Do not trust client-generated status changes.

The server remains authoritative.

---

33. Wallet & Billing

The Wallet/Billing feature owns:

- wallet balance
- credits
- transactions
- payment integration
- GST calculations
- invoices
- refunds
- payment status

Financial calculations must be deterministic.

Never use floating-point arithmetic for money when exact decimal representation is required.

Use appropriate decimal database types.

---

34. Razorpay

Razorpay integration must remain inside the Wallet/Billing feature.

Do not call Razorpay directly from unrelated modules.

The backend must verify payment state.

Never consider a payment successful merely because the frontend reports success.

Webhook signatures must be verified.

Payment records must be idempotent.

---

35. Email

Email functionality belongs to the appropriate mailer/shared infrastructure.

Use:

- NestJS Mailer
- Nodemailer
- SMTP

Email templates should be reusable.

Do not place large HTML email templates inside controllers or services.

---

36. Community

The Community feature owns:

- member directory
- profiles
- social feed
- posts
- comments
- interactions

Keep community-specific business logic inside this module.

Do not make other modules directly manipulate community repositories.

---

37. CMS

The CMS feature owns:

- pages
- sliders
- galleries
- banners
- content management
- site configuration

CMS content must be separated from application business logic.

Do not hardcode editable marketing content inside React components.

38. Dynamic Branding

Brand colors must use CSS design tokens.

Never hardcode colors such as:

```
bg-blue-600
text-purple-500
bg-red-500
```

when the color represents a brand/theme value.

Use:

```
bg-primary
text-primary
bg-secondary
text-secondary
bg-accent
text-accent
```

or CSS variables:

```
var(--brand-primary)
var(--brand-secondary)
var(--brand-accent)
var(--brand-card)
var(--brand-surface)
var(--brand-dark-surface)
var(--brand-dark-card)
```

The source specification explicitly requires theme tokens instead of hardcoded brand colors.

39. Dynamic Logo Color Synchronization

The active brand configuration must be loaded from the appropriate:

```
logo_settings
site_settings
```

configuration.

Inject active theme values into CSS variables during application initialization.

Do not duplicate brand values across dozens of components.

---

40. UI Design System

Use:

- shadcn/ui
- Magic UI
- HeroUI
- Tremor
- Framer Motion

Components should look like one cohesive design system.

Do not mix random component styles.

Avoid:

- inconsistent border radii
- random shadows
- arbitrary colors
- inconsistent spacing
- excessive gradients
- unnecessary animations

---

41. Accessibility

All UI must consider accessibility.

Use:

- semantic HTML
- keyboard navigation
- accessible labels
- focus states
- sufficient contrast
- ARIA only when necessary
- accessible dialogs
- accessible forms

Do not make a UI accessible only through mouse interaction.

42. Responsive Design

Every user-facing page must work across:

```
mobile
tablet
desktop
large desktop
```

Do not design desktop-only interfaces.

Meeting room and floor-map experiences must receive special consideration on smaller screens.

43. Loading States

Every asynchronous UI flow should have an appropriate loading state.

Use:

- skeletons
- loading indicators
- disabled action states

Avoid blank screens while data is loading.

44. Error States

Every important data-fetching page must handle:

```
loading
success
empty
error
```

Do not assume APIs always succeed.

Display user-friendly errors.

Do not expose backend stack traces to users.

45. Forms

Forms must have:

- clear labels
- validation
- error messages
- loading state
- disabled submit state where appropriate
- success feedback

Frontend validation improves UX.

Backend validation remains authoritative.

---

46. State Management

Do not introduce global state for everything.

Use:

- React state for local state
- URL state for shareable filters
- Server Components/server fetching where possible
- feature-specific stores for complex client state
- global state only when genuinely global

Keep feature state inside the feature.

---

47. API Client

Create a centralized API client.

Do not repeatedly implement:

```
fetch(...)
```

with different authentication/error logic throughout the application.

Centralize:

- base URL
- authentication
- headers

- JSON parsing
- error normalization
- timeout handling where applicable

---

48. Error Codes

Use stable machine-readable error codes.

Examples:

```
INVALID_CREDENTIALS
UNAUTHORIZED
FORBIDDEN
RESOURCE_NOT_FOUND
VALIDATION_ERROR
SLOT_ALREADY_BOOKED
ROOM_UNAVAILABLE
BOOKING_EXPIRED
INSUFFICIENT_BALANCE
PAYMENT_FAILED
PAYMENT_VERIFICATION_FAILED
INVALID_WEBHOOK
```

Do not make frontend logic depend on human-readable error messages.

---

49. Logging

Use structured logging.

Log:

- request identifiers
- important state transitions
- errors
- payment events
- booking events
- authentication events where appropriate

Never log:

- passwords
- JWT secrets
- payment secrets
- raw authorization tokens
- sensitive personal information unnecessarily

50. Testing

Every major feature should have tests.

Minimum expectations:

```
Unit Tests
Integration Tests
E2E Tests
```

Prioritize testing for:

- authentication
- authorization
- booking conflicts
- availability
- wallet transactions
- payment verification
- invoice calculations
- critical API endpoints

51. Testing Philosophy

Tests should verify behavior rather than implementation details.

Bad:

```
expect(privateMethod).toHaveBeenCalled()
```

Prefer:

```
expect(booking.status).toBe("CONFIRMED")
```

Test failure scenarios, not only successful scenarios.

52. Performance

Avoid unnecessary:

- database queries
- API requests
- client-side rendering
- large JavaScript bundles
- repeated calculations
- WebSocket broadcasts

Use pagination for potentially large collections.

Use database indexes for frequently queried fields.

---

53. Pagination

Large datasets must not be returned without pagination.

Examples:

- bookings
- members
- transactions
- invoices
- community posts
- CMS records

Prefer a consistent pagination contract.

Example:

```
{
  "items": [],
  "pagination": {
    "page": 1,
    "pageSize": 20,
    "total": 100,
    "totalPages": 5
  }
}
```

---

54. Search & Filtering

Search functionality should be implemented at the appropriate backend layer.

Do not fetch thousands of records to the browser and filter them in React.

Support:

- search
- filtering
- sorting
- pagination

where appropriate.

---

55. File Uploads

If file uploads are implemented:

- validate MIME type
- validate size
- sanitize filenames
- avoid trusting extensions
- store files outside application source
- use controlled URLs
- restrict access where necessary

Never allow arbitrary executable uploads.

---

56. Admin Architecture

Admin functionality should be protected by authorization.

Admin pages must not merely hide UI elements.

The backend must enforce authorization.

Example:

```
Admin UI
   ↓
Protected API
   ↓
Authorization Guard
   ↓
Service
   ↓
Database
```

---

57. Auditability

Important administrative and financial actions should be auditable.

Examples:

- booking cancellation
- booking modification
- refund
- wallet adjustment
- role change
- room configuration change
- CMS modification
- branding changes

Where appropriate, maintain audit records containing:

```
actor
action
resource
resourceId
timestamp
metadata
```

58. Git Rules

Use clear commits.

Prefer:

```
feat: add meeting room availability engine
fix: prevent overlapping bookings
refactor: isolate wallet billing module
docs: update API documentation
test: add booking conflict tests
```

Avoid:

```
update
changes
final
new
fix stuff
```

Do not commit generated build artifacts unless explicitly required.

59. Dependency Rules

Before adding a dependency:

1. determine whether an existing dependency already solves the problem
2. check whether the dependency is actively maintained
3. consider bundle/runtime impact
4. verify compatibility
5. document the reason if the dependency is significant

Do not install packages unnecessarily.

60. No Duplicate Implementations

Before creating a helper/component/service:

Search the repository.

If an equivalent implementation exists:

- reuse it
- improve it
- extract it
- or explain why a separate implementation is required

Do not create:

```
formatDate.ts
dateUtils.ts
dateHelper.ts
dateFormatter.ts
```

for the same responsibility.

61. Existing Code First

Before modifying a feature:

1. inspect the repository
2. understand the existing architecture
3. identify related files
4. identify existing utilities/components/services

5. determine dependencies
6. make the smallest coherent change

Never blindly overwrite existing working code.

---

62. Refactoring Rules

When refactoring:

- preserve behavior unless behavior change is intentional
- keep API compatibility where required
- update tests
- update documentation
- remove obsolete code
- avoid unrelated refactors

Do not mix a large architectural refactor with an unrelated feature unless necessary.

---

63. Agent Workflow

For every task, follow this sequence:

Step 1 — Understand

Read:

```
AGENTS.md
README.md
package.json
environment configuration
relevant feature files
database schema
```

Step 2 — Inspect

Search the repository for:

- existing implementations
- related services
- reusable components
- API endpoints
- types
- tests

Step 3 — Plan

Before modifying multiple files, identify:

- affected features
- dependencies
- database changes
- API changes
- UI changes
- testing requirements

Step 4 — Implement

Implement according to the architecture rules.

Step 5 — Validate

Run:

```
typecheck
lint
tests
build
```

where available.

Step 6 — Review

Check:

- feature isolation
- security
- type safety
- error handling
- accessibility
- responsiveness
- duplicate code
- unintended regressions

Step 7 — Report

Explain:

- what changed
- files affected
- database changes
- API changes
- tests performed

- remaining limitations

64. Agent Decision Rules

When uncertain:

Prefer existing code over new code.

Prefer feature-local code over global code.

Prefer typed interfaces over `any`.

Prefer backend enforcement over frontend assumptions.

Prefer database constraints over application-only validation.

Prefer reusable primitives over duplicated components.

Prefer explicit errors over silent failures.

Prefer small focused services over giant services.

Prefer Server Components when client interactivity is unnecessary.

65. Do Not

Never:

- use `any`
- expose secrets
- trust frontend payment confirmation
- trust frontend authorization
- put business logic in controllers
- put business logic directly in page components
- access another feature's private repository
- hardcode brand colors
- duplicate API clients
- duplicate business rules
- bypass validation
- disable TypeScript strictness
- ignore failing tests
- suppress errors without explanation
- commit `.env` secrets
- introduce dependencies without reason
- perform destructive database changes without migration

- silently change existing business behavior

66. Definition of Done

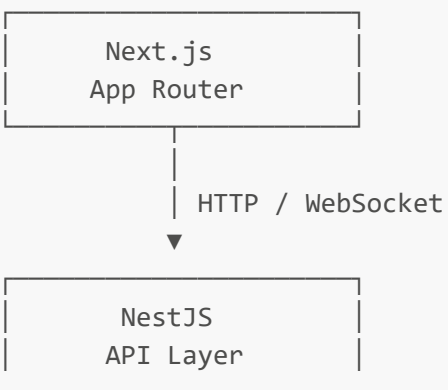
A task is not complete merely because the UI appears to work.

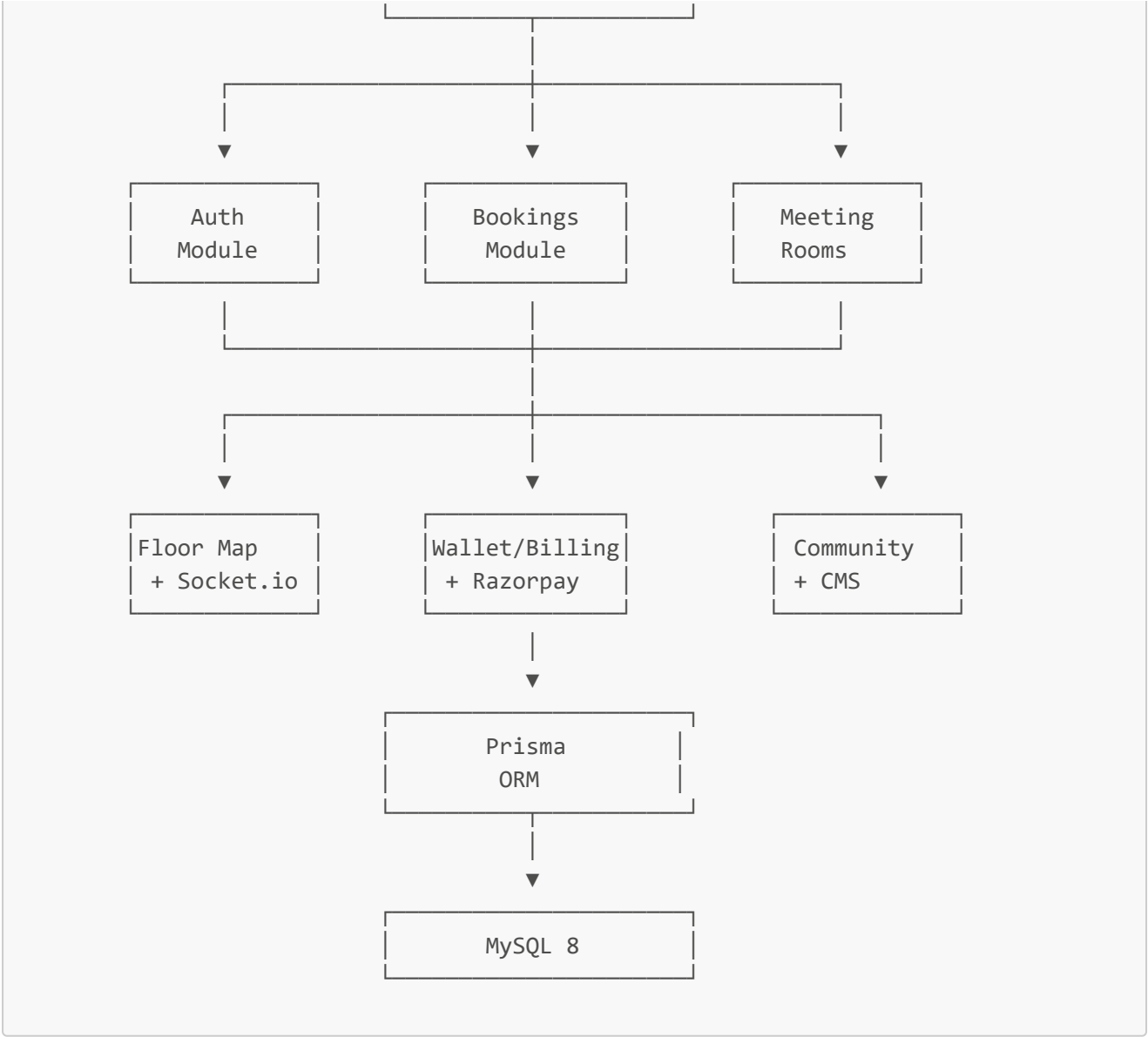
A feature is considered complete when:

- [ ] Architecture follows feature boundaries
- [ ] TypeScript is strict
- [ ] No `any` was introduced
- [ ] DTO/input validation exists
- [ ] Authorization is enforced where required
- [ ] Backend business logic is implemented
- [ ] Database changes use Prisma migrations
- [ ] Error handling is implemented
- [ ] API documentation is updated
- [ ] Loading states exist
- [ ] Error states exist
- [ ] Empty states exist where applicable
- [ ] Responsive behavior is verified
- [ ] Accessibility has been considered
- [ ] Tests are added/updated
- [ ] Lint passes
- [ ] Typecheck passes
- [ ] Build passes
- [ ] No secrets were committed
- [ ] No unnecessary dependencies were added
- [ ] No unrelated files were modified

67. Final Architecture

The final system should conceptually follow:





The architecture must remain modular enough that individual features can be independently rewritten, upgraded, tested, and maintained without introducing unnecessary side effects. This is a core requirement of the supplied workspace specification.

---

68. Priority Order

When requirements conflict, use this priority:

1. Security
2. Data integrity
3. Correct business behavior
4. Feature isolation
5. Type safety
6. Maintainability
7. Performance
8. Accessibility
9. UI consistency

10. Developer convenience

Never sacrifice security or data integrity merely to simplify implementation.

---

69. Source of Truth

For architecture and technology decisions, this `AGENTS.md` is the primary development guide.

When implementing a feature:

- follow these rules
- inspect existing code
- preserve existing valid behavior
- avoid unnecessary architectural deviation
- document intentional exceptions

If an existing implementation violates these rules, do not automatically rewrite the entire system.

Make targeted improvements as part of the relevant task.

---

70. Agent Instruction

You are working on a production-oriented coworking-space platform.

Do not behave like a code generator that simply produces files.

Behave like a senior full-stack engineer.

Before coding:

```
Understand → Inspect → Plan → Implement → Validate → Review
```

Every change must improve or preserve:

```
Correctness
Security
Architecture
Maintainability
Type Safety
User Experience
Performance
```

Do not claim a feature is complete until it has been implemented, integrated, and validated against the repository's actual codebase.